

Observability and Monitoring of LLM Models Hosted on vLLM

Armando Soares Sousa
Federal University of Piauí – UFPI
Teresina, Piauí, Brazil
armando@ufpi.edu.br

Abstract

The increasing adoption of applications based on Large Language Models (LLMs) has intensified the need for efficient infrastructures for model serving, inference, and continuous monitoring. Unlike traditional web applications, Generative AI systems exhibit specific operational characteristics, such as time to first token, token throughput, intensive GPU usage, key-value (KV) cache pressure, and performance variation according to prompt and response length. This paper presents an observability and monitoring prototype for LLMs served through vLLM, integrated with Prometheus for metric collection and Grafana for dashboard visualization. The analyzed solution uses a Docker Compose-based architecture, exposing an OpenAI-compatible API for question-answer interactions, a `/metrics` endpoint for metric export, and monitoring dashboards. This paper discusses the main architectural components, critical LLM inference metrics, the integrated operation of vLLM, Prometheus, and Grafana, and recommendations for evolving the prototype toward production environments. The results indicate that the integration between vLLM, Prometheus, and Grafana provides a practical and extensible foundation for observability in Generative AI applications.

Keywords

Generative AI, Large Language Models, vLLM, Prometheus, Grafana, Observability, Monitoring, Inference, MLOps, LLMops

1 Introduction

Generative AI applications based on Large Language Models (LLMs) have become central components in modern solutions for automated support, retrieval-augmented generation, summarization, text classification, institutional assistants, and intelligent agents. As these systems move from experimentation to production, it becomes necessary to monitor not only API availability, but also the internal behavior of the inference engine, GPU utilization, token throughput, user-perceived latency, and memory bottlenecks.

In this context, vLLM has emerged as a high-performance solution for LLM serving. Its main technical contribution is associated with *PagedAttention*, a mechanism for efficient management of the KV cache inspired by memory paging techniques used in operating systems. The original work reports that vLLM was designed to reduce memory waste in the KV cache and increase LLM inference throughput, especially under concurrent request workloads [1, 2].

However, simply hosting an LLM is not sufficient to ensure operational reliability. LLM-based systems may degrade due to long prompts, large generated outputs, request queues, GPU saturation, memory pressure, or increased time to first token. Therefore, observability becomes an essential requirement for understanding system behavior and supporting scalability, optimization, and maintenance decisions.

This paper presents an observability and monitoring prototype for an LLM hosted on vLLM and integrated with Prometheus and Grafana. Prometheus acts as the metric collector and time-series database, periodically scraping the `/metrics` endpoint exposed by vLLM. Grafana acts as the visualization layer, enabling dashboards for latency, time to first token, token throughput, active requests, queued requests, and KV cache usage. The official vLLM documentation states that the OpenAI-compatible server exposes production metrics through the `/metrics` endpoint, which enables direct integration with Prometheus [3, 4].

The main objective of this paper is to describe the architecture, components, metrics, and integrated operation of an observability solution for LLMs hosted on vLLM. As a contribution, the paper organizes a reference model for local or experimental deployment of a Generative AI monitoring stack and discusses limitations and improvements required for production environments.

2 Background

2.1 Large Language Models and Production Inference

Large Language Models are deep neural networks trained on large textual corpora to perform natural language understanding and generation tasks. In Generative AI applications, these models are frequently accessed through inference APIs, where a client sends a request containing messages, context, generation parameters, and token limits.

LLM inference differs from conventional application execution. In a traditional API, the response is often produced in a relatively predictable computational stage. In autoregressive LLMs, however, the answer is generated token by token. As a result, performance is influenced by prompt length, generated output length, concurrency, batch scheduling, GPU usage, and KV cache management.

Thus, traditional metrics such as average response time and HTTP error rate remain important, but they are not sufficient. LLM systems also require specific metrics such as *Time To First Token* (TTFT), input and output token throughput, end-to-end latency, request queue size, running requests, and GPU cache utilization.

2.2 vLLM and PagedAttention

vLLM is an inference engine for high-throughput LLM serving. Its central proposal is to reduce memory waste in the KV cache and improve throughput through *PagedAttention*. The original paper describes *PagedAttention* as an algorithm inspired by virtual memory paging, enabling block-based management of the KV cache and efficient memory sharing across requests [1, 2].

The KV cache stores intermediate keys and values used by the attention mechanism during autoregressive generation. Without efficient management, this cache may lead to fragmentation, memory

waste, and limited batch sizes. vLLM addresses this issue by managing memory in blocks, increasing the capacity to serve concurrent requests and improving GPU utilization.

In addition, vLLM provides an OpenAI-compatible server, allowing existing applications to use OpenAI-like clients with minimal changes, often requiring only the modification of the `base_url`, `model`, and `api_key` parameters.

2.3 Observability in Generative AI Systems

Observability is the ability to understand the internal state of a system from its external signals. In modern systems, these signals commonly involve metrics, logs, traces, and events. In Generative AI applications, observability must address both traditional software concerns and model inference-specific characteristics.

Relevant operational questions include whether the model is responding, whether latency is within the expected service level, whether the user receives the first token quickly, whether the request queue is growing, whether the GPU is saturated, whether the KV cache is close to its limit, and whether token throughput is adequate. These questions cannot be answered adequately by application logs alone. The inference server must expose specialized metrics.

2.4 Prometheus

Prometheus is an open-source monitoring and alerting platform based on time-series data. Its collection model is predominantly pull-based: the Prometheus server periodically queries HTTP metric endpoints exposed by monitored services. The official documentation describes the `scrape_config` block as the configuration unit that specifies scrape targets and parameters, using either static target definitions or service discovery mechanisms [5, 6].

In the analyzed prototype, Prometheus collects vLLM metrics from:

```
http://vllm:8000/metrics
```

The configured scrape interval is five seconds, enabling rapid observation of metric variations during inference tests.

2.5 Grafana

Grafana is a widely used visualization and operational analysis platform for dashboards. It provides native support for Prometheus as a data source, without requiring additional plugins [7]. Grafana also supports data source provisioning through YAML files, making it possible to treat visualization infrastructure as code [8].

3 Prototype Architecture

The analyzed solution is organized around three main components: vLLM, Prometheus, and Grafana. Deployment is performed through Docker Compose, which simplifies service networking, port mapping, environment configuration, and reproducibility.

Figure 1 summarizes the overall architecture. The client sends requests to the vLLM OpenAI-compatible API. vLLM performs model inference and exposes operational metrics. Prometheus scrapes these metrics periodically. Grafana queries Prometheus and presents dashboards.

The vLLM server is responsible for inference and metric export. Prometheus is responsible for metric collection, time-series storage, and PromQL query execution. Grafana is responsible for visualization and dashboard construction.

4 Prototype Description

4.1 Project Structure

The prototype contains the main files shown in Listing 1. The `docker-compose.yml` file defines the stack services. The `prometheus/` directory contains metric scraping configuration. The `grafana/` directory contains the automatic data source configuration. The files `teste.sh` and `teste.py` validate the question-answer interaction with the model hosted on vLLM.

Listing 1: Prototype directory structure.

```
vllm-server-prometheus-grafana/
|-- docker-compose.yml
|-- README.md
|-- teste.sh
|-- teste.py
|-- painel_grafana.md
|-- pyproject.toml
|-- uv.lock
|-- main.py
|-- prometheus/
|   |-- prometheus.yml
|-- grafana/
|   |-- provisioning/
|   |   |-- datasources/
|   |   |-- datasource.yml
```

4.2 vLLM Server

The vLLM service uses the following container image:

```
image: vllm/vllm-openai:v0.6.3
```

The default configured model is:

```
Qwen/Qwen2.5-0.5B-Instruct
```

The server is initialized with parameters such as `-model`, `-api-key`, `-gpu-memory-utilization`, `-max-model-len`, `-max-num-seqs`, `-dtype float16`, and `-enforce-eager`. These parameters control essential serving aspects. For instance, `-gpu-memory-utilization` defines the fraction of GPU memory available to vLLM. `-max-model-len` defines the maximum context length. `-max-num-seqs` controls the maximum number of simultaneous sequences, directly impacting concurrency and KV cache pressure.

4.3 Prometheus Configuration

The main Prometheus target is defined as:

```
scrape_configs:
- job_name: "vllm"
  metrics_path: /metrics
  static_configs:
  - targets: ["vllm:8000"]
```

This configuration instructs Prometheus to collect metrics from the `/metrics` endpoint of the vllm service using the container hostname available inside the Docker Compose network.

4.4 Grafana Configuration

Grafana is automatically configured through the file:

```
grafana/provisioning/datasources/datasource.yml
```

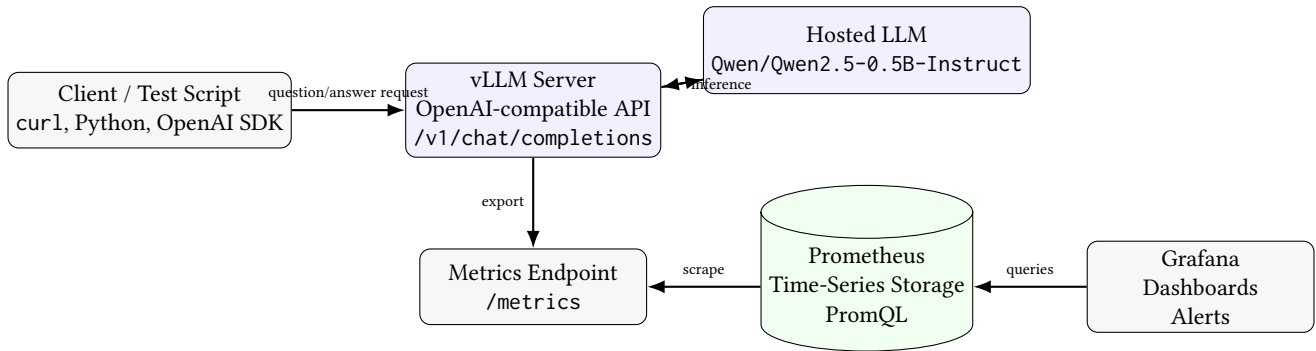


Figure 1: Reference architecture for observability and monitoring of an LLM hosted on vLLM with Prometheus and Grafana.

The data source definition points to Prometheus:

```

datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true

```

As a result, when the stack starts, Grafana already recognizes Prometheus as the default source for metric queries.

5 Metrics for LLM Monitoring

The vLLM documentation organizes its metrics into two main categories: server-level metrics, generally represented as gauges or counters, and request-level metrics, often represented as histograms. Request metrics are especially relevant for service-level objectives because they capture request size and timing characteristics [4].

Table 1 summarizes the main metrics used in the prototype and their operational interpretation.

5.1 End-to-End Latency

End-to-end latency measures the complete time between receiving the request and completing the response. In Prometheus, it can be queried using a histogram expression such as:

```

histogram_quantile(
  0.95,
  sum by (le) (
    rate({__name__=~"(vllm:|vllm_)e2e_request_latency_seconds_bucket"}[5m])
  )
)

```

This query computes the 95th percentile of latency, which is more informative for user experience than the mean alone.

5.2 Time To First Token

Time To First Token (TTFT) measures the time until the first token is produced. It is especially important in streaming applications because it strongly influences perceived responsiveness.

```

histogram_quantile(
  0.95,
  sum by (le) (
    rate({__name__=~"(vllm:|vllm_)time_to_first_token_seconds_bucket"}[5m])
  )
)

```

A high TTFT may indicate request queueing, long prompts, bottlenecks during prefill, or server saturation.

5.3 Token Throughput

Token throughput is one of the most important metrics for evaluating the actual capacity of an LLM server. It can be separated into input prompt tokens and generated output tokens.

```
sum(rate({__name__=~"(vllm:|vllm_)generation_tokens_total"}[1m]))
```

```
sum(rate({__name__=~"(vllm:|vllm_)prompt_tokens_total"}[1m]))
```

These metrics help determine whether the system is being stressed by long prompts, long responses, or high concurrency.

5.4 Running and Waiting Requests

Running and waiting request metrics are important to detect saturation:

```
max({__name__=~"(vllm:|vllm_)num_requests_running"})
```

```
max({__name__=~"(vllm:|vllm_)num_requests_waiting"})
```

When `num_requests_waiting` grows persistently, it strongly suggests that demand exceeds the processing capacity of the server.

5.5 KV Cache Usage

The KV cache is critical for autoregressive inference. In vLLM, its usage can be monitored through metrics such as:

```
max({__name__=~"(vllm:|vllm_)gpu_cache_usage_perc"})
```

High KV cache usage may indicate the need to reduce the maximum context length, reduce concurrency, limit output tokens, or use GPUs with more memory.

6 Integrated Operation

6.1 Inference Flow

The inference flow occurs as follows: the user or client script sends a question; the HTTP request is sent to `/v1/chat/completions`; vLLM processes the prompt and performs generation; the model returns the generated response; vLLM updates its internal metrics; and the metrics become available at `/metrics`.

An example request is shown below:

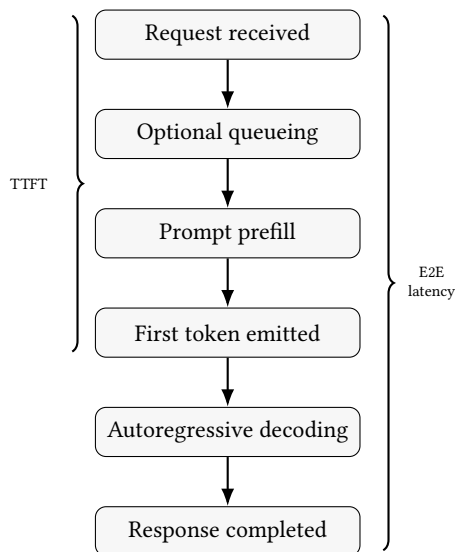
```

curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer local-dev-key" \
-d '{
  "model": "Qwen/Qwen2.5-0.5B-Instruct",

```

Table 1: Core metrics for monitoring LLM inference with vLLM.

Metric	Type	Purpose	Operational Interpretation
e2e_request_latency_seconds_buster	Histogram	End-to-end request latency	High p95 indicates degraded user experience or excessive generation time.
time_to_first_token_seconds_buster	Histogram	Time until the first generated token	High TTFT indicates queueing, long prompt prefill, or saturation.
generation_tokens_total	Counter	Output token throughput	Low throughput under load indicates bottlenecks or conservative serving configuration.
prompt_tokens_total	Counter	Input token throughput	High prompt token rate may indicate long contexts or RAG-heavy workloads.
num_requests_running	Gauge	Active requests being processed	High values indicate high concurrency and active GPU scheduling.
num_requests_waiting	Gauge	Queued requests waiting for processing	Persistent growth indicates saturation or insufficient capacity.
gpu_cache_usage_perc	Gauge	KV cache usage on GPU	High values indicate memory pressure and risk of reduced capacity or out-of-memory errors.

**Figure 2: Relationship between request lifecycle, TTFT, and end-to-end latency.**

```

{
  "messages": [
    { "role": "system", "content": "You are a helpful assistant." },
    { "role": "user", "content": "Explain Generative AI." }
  ],
  "max_tokens": 512,
  "temperature": 0.7
}

```

6.2 Metric Collection Flow

After executing inference requests, vLLM exposes metrics at `/metrics`. Prometheus periodically collects these metrics:

```
Prometheus -> http://vllm:8000/metrics
```

The scrape interval configured in the prototype is:

```
scrape_interval: 5s
```

Table 2: Recommended Grafana panels for LLM monitoring.

Panel	Purpose
E2E latency p95	Evaluate total user-perceived response time.
TTFT p95	Evaluate initial responsiveness, especially for streaming.
Output tokens/s	Measure generation throughput.
Prompt tokens/s	Measure input workload and context pressure.
Running requests	Observe active concurrency.
Waiting requests	Detect queueing and saturation.
KV cache usage	Detect GPU memory pressure.

This interval is suitable for local experimentation because it allows the operator to quickly observe changes in time-series data during tests.

6.3 Visualization Flow

Grafana queries Prometheus using PromQL and renders visual panels. Table 2 lists useful dashboard panels for LLM observability.

7 Execution Procedure

7.1 Prerequisites

The solution requires Docker, Docker Compose, an NVIDIA GPU compatible with CUDA, the NVIDIA driver, the NVIDIA Container Toolkit, and internet access to download the model. GPU access from Docker can be validated using:

```
docker run --rm --gpus all \
  nvidia/cuda:12.2.0-base-ubuntu22.04 nvidia-smi
```

7.2 Initial Configuration

The project directory can be accessed with:

```
cd vllm-server-prometheus-grafana
```

An optional `.env` file may define:

```
MODEL_ID=Qwen/Qwen2.5-0.5B-Instruct
```

```
VLLM_API_KEY=local-dev-key
GPU_MEM_UTIL=0.75
MAX_MODEL_LEN=4096
MAX_NUM_SEQS=64
GRAFANA_USER=admin
GRAFANA_PASS=admin
HF_TOKEN=
```

7.3 Starting the Stack

The full stack can be started with:

```
docker compose up -d
```

The containers can be checked with:

```
docker compose ps
```

vLLM logs can be followed using:

```
docker compose logs -f vllm
```

7.4 Validation

The vLLM model listing endpoint can be tested with:

```
curl -s http://localhost:8000/v1/models
```

Prometheus can be accessed at `http://localhost:9090`. The target vllm should appear as UP. A basic PromQL validation query is:

```
up{job="vllm"}
```

Grafana can be accessed at `http://localhost:3000`. The Prometheus data source should be automatically configured and used to import or build dashboards.

8 Discussion

The analyzed solution demonstrates that LLM observability must be designed specifically for inference workloads. In traditional systems, a healthy API can often be initially evaluated through availability, latency, and error rate. In LLM applications, however, the API may be available while the user experience is degraded due to high TTFT, low token throughput, or a growing request queue.

vLLM facilitates this process by exposing native metrics through `/metrics`, allowing consolidated tools such as Prometheus and Grafana to be integrated without additional instrumentation in the client application. This characteristic is especially relevant for teams adopting LLMops or MLOps practices in local, academic, corporate, or governmental environments.

Another relevant point is the diagnostic utility of dashboards. For example, if p95 latency increases while `num_requests_waiting` also grows, the issue is likely related to saturation. If KV cache usage approaches its limit, it may be necessary to reduce `MAX_MODEL_LEN`, limit response size, or reduce the number of simultaneous sequences. If TTFT grows without significant queue growth, the bottleneck may be related to prefill, prompt size, or model configuration.

Despite being functional, the prototype still has limitations. It monitors vLLM metrics, but it does not include detailed GPU metrics such as VRAM utilization, temperature, power consumption, or streaming multiprocessor utilization. It also does not include centralized logs, distributed traces, automated alerts, or protection for sensitive endpoints. In production environments, the solution

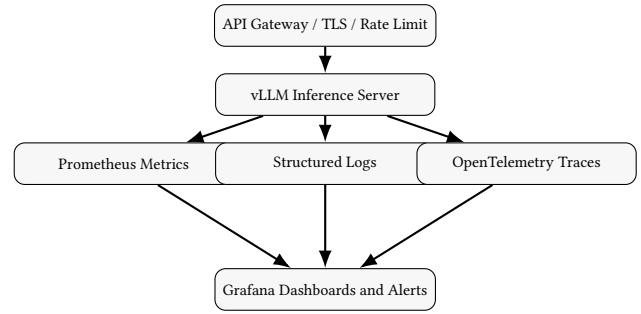


Figure 3: Possible production evolution combining metrics, logs, traces, gateway protection, and alerting.

should be complemented with tools such as NVIDIA DCGM Exporter, Loki, OpenTelemetry, Alertmanager, authentication, TLS, access control, and dashboard provisioning as code.

9 Threats to Validity

This paper presents a technical study based on a local prototype rather than a large-scale experimental evaluation. Therefore, several threats to validity should be considered.

First, the results and recommendations depend on the vLLM version used. Metric names and internal behavior may change across versions, so dashboards and PromQL queries should be validated whenever the server is upgraded. Second, the model used in the prototype is relatively small, which facilitates local execution but does not necessarily represent the behavior of larger models in production environments. Third, the absence of systematic load testing limits quantitative conclusions about throughput, latency, and scalability.

Despite these limitations, the prototype is suitable as a foundation for architectural understanding and initial validation of observability in LLM serving.

10 Recommendations for Evolution

The solution can be improved through several directions:

- (1) Update and pin validated versions of vLLM, Prometheus, and Grafana.
- (2) Add GPU monitoring with NVIDIA DCGM Exporter.
- (3) Create alerting rules for unavailability, high queue size, high latency, high TTFT, and KV cache pressure.
- (4) Provision dashboards as code, avoiding manual import.
- (5) Add structured logs for requests, errors, and response times.
- (6) Integrate OpenTelemetry for traces across the client application, gateway, and vLLM.
- (7) Protect sensitive endpoints, especially `/metrics`, Grafana, and the inference API.
- (8) Perform controlled load tests to define service-level objectives and operational limits.
- (9) Add a gateway layer with authentication, rate limiting, and TLS.
- (10) Evaluate horizontal scalability, especially in multi-GPU or multi-replica scenarios.

11 Conclusion

This paper presented an observability and monitoring solution for LLMs hosted on vLLM and integrated with Prometheus and Grafana. The solution validates the complete cycle of a Generative AI application: question submission, model inference, metric exposure, Prometheus collection, and Grafana visualization.

The main contribution of the prototype is to demonstrate, in a practical way, how LLM-serving-specific metrics can be incorporated into the operation of Generative AI systems. Metrics such as end-to-end latency, TTFT, tokens per second, running requests, waiting requests, and KV cache usage provide a more appropriate operational view of LLM behavior than traditional metrics alone.

The integration between vLLM, Prometheus, and Grafana constitutes a solid foundation for observability practices in Generative AI. For production environments, however, the solution should be extended with security, alerting, GPU monitoring, logs, traces, and systematic load testing.

Acknowledgments

This paper was derived from the analysis of a prototype for validating LLM serving with vLLM, Prometheus, and Grafana. The author

thanks the open-source communities behind vLLM, Prometheus, and Grafana for their documentation and tools.

References

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 611–626. <https://doi.org/10.1145/3600006.3613165>
- [2] W. Kwon et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint arXiv:2309.06180*. <https://arxiv.org/abs/2309.06180>
- [3] vLLM Project. 2026. Production Metrics – vLLM Documentation. Retrieved May 23, 2026 from <https://docs.vllm.ai/en/stable/usage/metrics/>
- [4] vLLM Project. 2026. Metrics – vLLM Documentation. Retrieved May 23, 2026 from <https://docs.vllm.ai/en/stable/design/metrics/>
- [5] Prometheus Authors. 2026. Prometheus Configuration Documentation. Retrieved May 23, 2026 from <https://prometheus.io/docs/prometheus/latest/configuration/configuration/>
- [6] Prometheus Authors. 2026. Getting Started with Prometheus. Retrieved May 23, 2026 from https://prometheus.io/docs/prometheus/latest/getting_started/
- [7] Grafana Labs. 2026. Configure the Prometheus Data Source. Retrieved May 23, 2026 from <https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/>
- [8] Grafana Labs. 2026. Grafana Data Sources and Provisioning. Retrieved May 23, 2026 from <https://grafana.com/docs/grafana/latest/datasources/>
- [9] OpenTelemetry Authors. 2026. What is OpenTelemetry? Retrieved May 23, 2026 from <https://opentelemetry.io/docs/what-is-opentelemetry/>